```
// app.js — lógica de frontend (vanilla JS, sin frameworks)

const state = {
  user: null,
  products: [],
  suppliers: [],
  clients: [],
  currentClientId: null
};

const PAYMENT_LABELS = {
  efectivo: 'Efectivo',
  debito: 'Débito',
  credito: 'Crédito',
  transferencia: 'Transferencia',
  cuenta_corriente: 'Cuenta corriente'
};

const NAV_ITEMS = [
  { id: 'dashboard', label: 'Inicio', roles: ['duena', 'empleada'] },
  { id: 'venta', label: 'Venta diaria', roles: ['duena', 'empleada'] },
  { id: 'historial', label: 'Historial', roles: ['duena', 'empleada'] },
  { id: 'stock', label: 'Stock', roles: ['duena', 'empleada'] },
  { id: 'cc', label: 'Cuentas corrientes', roles: ['duena', 'empleada'] },
  { id: 'proveedores', label: 'Proveedores', roles: ['duena'] },
  { id: 'movimientos', label: 'Movimientos', roles: ['duena'] },
  { id: 'gastos', label: 'Gastos', roles: ['duena'] },
  { id: 'reportes', label: 'Reportes', roles: ['duena'] },
  { id: 'usuarios', label: 'Usuarias', roles: ['duena'] }
];

const CATEGORIES = [
  'Accesorio', 'Biker', 'Buzos', 'Camperas', 'Capri', 'Catsuit corto', 'Catsuit
largo',
  'Leggins', 'Manga larga', 'Musculosas', 'Pantalones', 'Remeras', 'Short', 'Short
pollera',
  'Top', 'Vestidos'
];

// Arma las <option> de categoría. Si la prenda ya tenía cargada una categoría que
no
// está en la lista fija (ej. algo viejo escrito distinto), la deja como primera
opción
// para no perder el dato — pero lo ideal es pasarla a una de las categorías de la
```

```
lista.
function categoryOptionsHtml(selected) {
  const extra = selected && !CATEGORIES.includes(selected) ? [selected] : [];
  const opts = [...extra, ...CATEGORIES];
  return '<option value="">—</option>' + opts.map((c) => `<option value="${c}" ${c
=== selected ? 'selected' : ''}>${c}</option>`).join('');
}

// ---------- helpers ----------
// Saca tildes/acentos y pasa a minúsculas, para que buscar "pantalon" encuentre
"Pantalón".
function normalize(str) {
  return (str || '')
    .toString()
    .normalize('NFD')
    .replace(/[̀-ͯ]/g, '')
    .toLowerCase();
}

async function api(url, options = {}) {
  const res = await fetch(url, {
    method: options.method || 'GET',
    headers: { 'Content-Type': 'application/json' },
    credentials: 'same-origin',
    body: options.body ? JSON.stringify(options.body) : undefined
  });
  const isJson = res.headers.get('content-type')?.includes('application/json');
  const data = isJson ? await res.json() : null;
  if (!res.ok) throw new Error((data && data.error) || 'Error de red');
  return data;
}

function money(n) {
  return '$' + Number(n || 0).toLocaleString('es-AR', { minimumFractionDigits: 2,
maximumFractionDigits: 2 });
}

function fmtDate(iso) {
  const d = new Date(iso);
  return d.toLocaleDateString('es-AR') + ' ' + d.toLocaleTimeString('es-AR', {
hour: '2-digit', minute: '2-digit' });
}

function el(html) {
  const t = document.createElement('template');
  t.innerHTML = html.trim();
  return t.content.firstChild;
```

```
}

function showModal(title, bodyEl) {
  document.getElementById('modal-title').textContent = title;
  const body = document.getElementById('modal-body');
  body.innerHTML = '';
  body.appendChild(bodyEl);
  document.getElementById('modal-backdrop').hidden = false;
}
function closeModal() {
  document.getElementById('modal-backdrop').hidden = true;
}
document.getElementById('modal-close').addEventListener('click', closeModal);
document.getElementById('modal-backdrop').addEventListener('click', (e) => {
  if (e.target.id === 'modal-backdrop') closeModal();
});

// ---------- AUTH ----------
document.getElementById('login-form').addEventListener('submit', async (e) => {
  e.preventDefault();
  const username = document.getElementById('login-username').value.trim();
  const password = document.getElementById('login-password').value;
  const errEl = document.getElementById('login-error');
  errEl.hidden = true;
  try {
    const { user } = await api('/api/login', { method: 'POST', body: { username,
password } });
    state.user = user;
    enterApp();
  } catch (err) {
    errEl.textContent = err.message;
    errEl.hidden = false;
  }
});

document.getElementById('logout-btn').addEventListener('click', async () => {
  await api('/api/logout', { method: 'POST' });
  state.user = null;
  document.getElementById('app').hidden = true;
  document.getElementById('login-screen').hidden = false;
});

async function checkSession() {
  const { user } = await api('/api/me');
  if (user) {
    state.user = user;
    enterApp();
```

```
  }
}

function enterApp() {
  document.getElementById('login-screen').hidden = true;
  document.getElementById('app').hidden = false;
  document.getElementById('user-name').textContent = `${state.user.name}
(${state.user.role === 'duena' ? 'Dueña' : 'Empleada'})`;
  buildNav();
  showView('dashboard');
}

function buildNav() {
  const nav = document.getElementById('main-nav');
  nav.innerHTML = '';
  NAV_ITEMS.filter((i) => i.roles.includes(state.user.role)).forEach((item) => {
    const btn = document.createElement('button');
    btn.textContent = item.label;
    btn.dataset.view = item.id;
    btn.addEventListener('click', () => showView(item.id));
    nav.appendChild(btn);
  });
}

const VIEW_LOADERS = {
  dashboard: loadDashboard,
  venta: loadVentaView,
  historial: loadHistorial,
  stock: loadStock,
  proveedores: loadSuppliers,
  movimientos: loadMovimientos,
  gastos: loadGastos,
  cc: loadClientes,
  reportes: loadReportes,
  usuarios: loadUsuarios
};

function showView(id) {
  document.querySelectorAll('.view').forEach((v) => v.classList.remove('active'));
  document.getElementById('view-' + id).classList.add('active');
  document.querySelectorAll('#main-nav button').forEach((b) =>
b.classList.toggle('active', b.dataset.view === id));
  if (VIEW_LOADERS[id]) VIEW_LOADERS[id]();
}

// ---------- DASHBOARD ----------
async function loadDashboard() {
```

```
  const d = await api('/api/dashboard');
  const cards = document.getElementById('dashboard-cards');
  cards.innerHTML = '';
  cards.appendChild(cardEl('Ventas de hoy', money(d.todayTotal)));
  cards.appendChild(cardEl('Cantidad de ventas hoy', d.todayCount));
  if (d.pendingCC !== undefined) cards.appendChild(cardEl('Cuentas corrientes
pendientes', money(d.pendingCC)));
}

function cardEl(label, value) {
  return el(`<div class="card"><div class="card-label">${label}</div><div
class="card-value">${value}</div></div>`);
}

// ---------- STOCK ----------
async function refreshProducts() {
  state.products = await api('/api/products');
  return state.products;
}

async function loadStock() {
  await refreshProducts();
  const isOwner = state.user.role === 'duena';
  document.getElementById('new-product-btn').hidden = !isOwner;
  document.getElementById('new-product-batch-btn').hidden = !isOwner;
  document.getElementById('stock-filter-supplier-wrap').hidden = !isOwner;

  const theadRow = document.getElementById('products-thead-row');
  theadRow.innerHTML = isOwner
    ? '<th>Prenda</th><th>Categoría</th><th>Talle</th><th>Color</th><th>Costo</th>
<th>Venta</th><th>Stock</th><th>Proveedor</th><th></th>'
    : '<th>Prenda</th><th>Categoría</th><th>Talle</th><th>Color</th><th>Venta</th>
<th>Stock</th>';

  const categories = Array.from(new Set(state.products.map((p) =>
p.category).filter(Boolean))).sort();
  const catSelect = document.getElementById('stock-filter-category');
  const prevCat = catSelect.value;
  catSelect.innerHTML = '<option value="">Todas las categorías</option>' +
categories.map((c) => `<option value="${c}">${c}</option>`).join('');
  catSelect.value = categories.includes(prevCat) ? prevCat : '';

  if (isOwner) {
    if (state.suppliers.length === 0) state.suppliers = await
api('/api/suppliers');
    const supSelect = document.getElementById('stock-filter-supplier');
    const prevSup = supSelect.value;
```

```
    supSelect.innerHTML = '<option value="">Todos los proveedores</option>' +
state.suppliers.map((s) => `<option value="${s.id}">${s.name}</option>`).join('');
    if (Array.from(supSelect.options).some((o) => o.value === prevSup))
supSelect.value = prevSup;
  }

  renderProductsTable();
}

function renderProductsTable() {
  const isOwner = state.user.role === 'duena';
  const search = normalize(document.getElementById('stock-search').value.trim());
  const category = document.getElementById('stock-filter-category').value;
  const supplierId = isOwner ? document.getElementById('stock-filter-
supplier').value : '';

  let list = state.products;
  if (search) {
    list = list.filter((p) =>
      normalize(p.name).includes(search) ||
      normalize(p.category).includes(search) ||
      normalize(p.color).includes(search) ||
      normalize(p.size).includes(search) ||
      normalize(p.piece).includes(search)
    );
  }
  if (category) list = list.filter((p) => p.category === category);
  if (supplierId) list = list.filter((p) => String(p.supplierId || '') ===
supplierId);

  list = [...list].sort((a, b) =>
    normalize(a.name).localeCompare(normalize(b.name), 'es', { sensitivity: 'base'
}) ||
    normalize(a.size || '').localeCompare(normalize(b.size || ''), 'es', {
sensitivity: 'base' })
  );

  const tbody = document.querySelector('#products-table tbody');
  tbody.innerHTML = '';

  if (list.length === 0) {
    tbody.appendChild(el(`<tr><td colspan="${isOwner ? 9 : 6}" style="text-
align:center;color:var(--gray);">No hay prendas que coincidan con la búsqueda.
</td></tr>`));
    return;
  }
```

```
  list.forEach((p) => {
    const stockBadgeClass = p.stock <= p.lowStockThreshold ? 'badge-low' : 'badge-
ok';
    const stockCell = isOwner
      ? `<span class="stock-adjust-controls">
          <button type="button" class="btn btn-ghost btn-sm stock-step" data-
pid="${p.id}" data-delta="-1" title="Restar 1" ${p.stock <= 0 ? 'disabled' :
''}>−</button>
          <span class="badge ${stockBadgeClass}" data-stock-
badge="${p.id}">${p.stock}</span>
          <button type="button" class="btn btn-ghost btn-sm stock-step" data-
pid="${p.id}" data-delta="1" title="Sumar 1">+</button>
        </span>`
      : `<span class="badge ${stockBadgeClass}">${p.stock}</span>`;
    const supplierName = state.suppliers.find((s) => s.id === p.supplierId)?.name
|| '—';
    const nameCell = p.setName && p.setName !== p.name
      ? `${p.name}<br><small style="color:var(--gray);">${p.setName}</small>`
      : p.name;
    const talleCell = p.piece ? (p.size ? `${p.piece} · ${p.size}` : p.piece) :
(p.size || '—');
    const row = isOwner
      ? `<tr>
          <td>${nameCell}</td><td>${p.category || '—'}</td><td>${talleCell}</td>
<td>${p.color || '—'}</td>
          <td>${money(p.costPrice)}</td><td>${money(p.salePrice)}</td>
<td>${stockCell}</td><td>${supplierName}</td>
          <td>
            <button class="btn btn-ghost btn-sm" data-
edit="${p.id}">Editar</button>
            <button class="btn btn-danger btn-sm" data-
del="${p.id}">Borrar</button>
          </td>
        </tr>`
      : `<tr>
          <td>${nameCell}</td><td>${p.category || '—'}</td><td>${talleCell}</td>
<td>${p.color || '—'}</td>
          <td>${money(p.salePrice)}</td><td>${stockCell}</td>
        </tr>`;
    tbody.appendChild(el(row));
  });

  if (isOwner) {
    tbody.querySelectorAll('.stock-step').forEach((b) =>
      b.addEventListener('click', () => quickStockAdjust(Number(b.dataset.pid),
Number(b.dataset.delta)))
    );
```

```
    tbody.querySelectorAll('[data-edit]').forEach((b) =>
b.addEventListener('click', () => openProductForm(Number(b.dataset.edit))));
    tbody.querySelectorAll('[data-del]').forEach((b) =>
      b.addEventListener('click', async () => {
        if (!confirm('¿Borrar esta prenda del stock?')) return;
        await api(`/api/products/${b.dataset.del}`, { method: 'DELETE' });
        loadStock();
      })
    );
  }
}

const stockAdjustBusy = new Set();

async function quickStockAdjust(id, delta) {
  if (stockAdjustBusy.has(id)) return;
  const p = state.products.find((x) => x.id === id);
  if (!p || (delta < 0 && p.stock <= 0)) return;
  stockAdjustBusy.add(id);
  try {
    await api('/api/movements', {
      method: 'POST',
      body: {
        productId: id,
        type: delta > 0 ? 'ingreso' : 'egreso',
        qty: 1,
        reason: delta > 0 ? 'Ajuste rápido (+1)' : 'Ajuste rápido (-1)'
      }
    });
    p.stock += delta;
    renderProductsTable();
  } catch (err) {
    alert(err.message);
  } finally {
    stockAdjustBusy.delete(id);
  }
}

document.getElementById('stock-search').addEventListener('input',
renderProductsTable);
document.getElementById('stock-filter-category').addEventListener('change',
renderProductsTable);
document.getElementById('stock-filter-supplier').addEventListener('change',
renderProductsTable);

document.getElementById('new-product-btn').addEventListener('click', () =>
openProductForm(null));
```

```
document.getElementById('new-product-batch-btn').addEventListener('click', () =>
openProductBatchForm());

let batchRowId = 0;

function addBatchVariantRow(container, piece = '', size = '', stock = 1, price =
'') {
  ++batchRowId;
  const row = el(`
    <div class="batch-variant-row">
      <input type="text" class="batch-piece" placeholder="Pieza (ej: Top)"
value="${piece}" />
      <input type="text" class="batch-size" placeholder="Talle (ej: M)"
value="${size}" />
      <input type="number" class="batch-stock" min="0" value="${stock}"
placeholder="Cant." />
      <input type="number" class="batch-price" step="0.01" placeholder="Precio
(opcional)" value="${price}" />
      <button type="button" class="btn btn-ghost btn-sm batch-variant-remove"
title="Quitar">×</button>
    </div>
  `);
  container.appendChild(row);
  updateBatchRemoveButtonsState(container);
}

function updateBatchRemoveButtonsState(container) {
  const rows = container.querySelectorAll('.batch-variant-row');
  rows.forEach((r) => { r.querySelector('.batch-variant-remove').disabled =
rows.length <= 1; });
}

async function openProductBatchForm() {
  if (state.suppliers.length === 0) state.suppliers = await api('/api/suppliers');
  const supplierOptions = state.suppliers.map((s) => `<option
value="${s.id}">${s.name}</option>`).join('');

  const form = el(`
    <form class="form-card" style="max-width:none;box-shadow:none;padding:0;">
      <p style="margin-top:0;color:var(--gray);font-size:13px;">
        Usalo para cargar de una vez varios talles de la misma prenda, o las
piezas de un conjunto
        (ej: Top + Campera + Legging) —cada una con sus propios talles si hace
falta—, que después vas a poder
        vender por separado igual que cualquier otra prenda.
      </p>
      <label>Nombre base <input name="name" required placeholder="Ej: Musculosa
```

```
Blagnac negra" /></label>
      <div class="grid-2">
        <label>Categoría <select name="category">${categoryOptionsHtml('')}
</select></label>
        <label>Color <input name="color" /></label>
      </div>
      <div class="grid-2">
        <label>Precio costo <input name="costPrice" type="number" step="0.01"
min="0" /></label>
        <label>Precio venta (por defecto) <input name="salePrice" type="number"
step="0.01" min="0" /></label>
      </div>
      <div class="grid-2">
        <label>Alerta stock bajo <input name="lowStockThreshold" type="number"
min="0" value="3" /></label>
        <label>Proveedor <select name="supplierId"><option value="">—
</option>${supplierOptions}</select></label>
      </div>

      <div class="batch-generator">
        <label style="margin-bottom:4px;">Generador rápido (opcional)</label>
        <p style="margin:0 0 8px;color:var(--gray);font-size:12.5px;">
          Si es un conjunto, escribí las piezas. Si tiene talles, escribilos
también — se van a combinar
          (ej: piezas "Top, Legging" + talles "S, M" genera Top-S, Top-M, Legging-
S, Legging-M).
          Dejá uno de los dos vacío si no aplica.
        </p>
        <div class="grid-2">
          <label>Piezas, separadas por coma <input type="text" id="batch-gen-
pieces" placeholder="Ej: Top, Campera, Legging" /></label>
          <label>Talles, separados por coma <input type="text" id="batch-gen-
sizes" placeholder="Ej: S, M, L" /></label>
        </div>
        <button type="button" id="batch-generate-btn" class="btn btn-secondary
btn-sm">Generar filas</button>
      </div>

      <label>Prendas a crear (revisá / ajustá cantidad y precio de cada una)
</label>
      <div id="batch-variants"></div>
      <button type="button" id="batch-add-variant-btn" class="btn btn-ghost btn-
sm">+ Agregar fila manual</button>
      <p class="error-text" hidden></p>
      <button type="submit" class="btn btn-primary btn-block">Crear
prendas</button>
    </form>
```

```
  `);

  const variantsContainer = form.querySelector('#batch-variants');
  batchRowId = 0;
  addBatchVariantRow(variantsContainer);

  form.querySelector('#batch-add-variant-btn').addEventListener('click', () =>
addBatchVariantRow(variantsContainer));
  form.querySelector('#batch-generate-btn').addEventListener('click', () => {
    const pieces = form.querySelector('#batch-gen-
pieces').value.split(',').map((s) => s.trim()).filter(Boolean);
    const sizes = form.querySelector('#batch-gen-sizes').value.split(',').map((s)
=> s.trim()).filter(Boolean);
    let combos = [['', '']];
    if (pieces.length && sizes.length) {
      combos = [];
      pieces.forEach((p) => sizes.forEach((s) => combos.push([p, s])));
    } else if (pieces.length) {
      combos = pieces.map((p) => [p, '']);
    } else if (sizes.length) {
      combos = sizes.map((s) => ['', s]);
    }
    variantsContainer.innerHTML = '';
    combos.forEach(([p, s]) => addBatchVariantRow(variantsContainer, p, s));
  });
  variantsContainer.addEventListener('click', (e) => {
    if (e.target.classList.contains('batch-variant-remove')) {
      const rows = variantsContainer.querySelectorAll('.batch-variant-row');
      if (rows.length <= 1) return;
      e.target.closest('.batch-variant-row').remove();
      updateBatchRemoveButtonsState(variantsContainer);
    }
  });

  form.addEventListener('submit', async (e) => {
    e.preventDefault();
    const fd = new FormData(form);
    const entries = Object.fromEntries(fd.entries());
    const base = {
      name: entries.name,
      category: entries.category,
      color: entries.color,
      costPrice: entries.costPrice,
      salePrice: entries.salePrice,
      lowStockThreshold: entries.lowStockThreshold,
      supplierId: entries.supplierId
    };
```

```
    const variants = Array.from(variantsContainer.querySelectorAll('.batch-
variant-row')).map((row) => ({
      piece: row.querySelector('.batch-piece').value.trim(),
      size: row.querySelector('.batch-size').value.trim(),
      stock: Number(row.querySelector('.batch-stock').value) || 0,
      salePrice: row.querySelector('.batch-price').value
    }));
    const errEl = form.querySelector('.error-text');
    try {
      await api('/api/products/batch', { method: 'POST', body: { ...base, variants
} });
      closeModal();
      loadStock();
    } catch (err) {
      errEl.textContent = err.message;
      errEl.hidden = false;
    }
  });

  showModal('Carga múltiple de prendas', form);
}

async function openProductForm(id) {
  if (state.suppliers.length === 0 && state.user.role === 'duena') {
    state.suppliers = await api('/api/suppliers');
  }
  const p = id ? state.products.find((x) => x.id === id) : null;
  const supplierOptions = state.suppliers.map((s) => `<option value="${s.id}"
${p?.supplierId === s.id ? 'selected' : ''}>${s.name}</option>`).join('');

  const form = el(`
    <form class="form-card" style="max-width:none;box-shadow:none;padding:0;">
      <label>Nombre <input name="name" required value="${p?.name || ''}" />
</label>
      <div class="grid-2">
        <label>Categoría <select name="category">${categoryOptionsHtml(p?.category
|| '')}</select></label>
        <label>Color <input name="color" value="${p?.color || ''}" /></label>
      </div>
      <div class="grid-2">
        <label>Pieza (si es parte de un conjunto) <input name="piece"
value="${p?.piece || ''}" placeholder="Ej: Top" /></label>
        <label>Talle <input name="size" value="${p?.size || ''}" /></label>
      </div>
      <label>Stock <input name="stock" type="number" min="0" value="${p?.stock ??
0}" required /></label>
      <div class="grid-2">
```

```
        <label>Precio costo <input name="costPrice" type="number" step="0.01"
min="0" value="${p?.costPrice ?? ''}" /></label>
        <label>Precio venta <input name="salePrice" type="number" step="0.01"
min="0" value="${p?.salePrice ?? ''}" required /></label>
      </div>
      <div class="grid-2">
        <label>Alerta stock bajo <input name="lowStockThreshold" type="number"
min="0" value="${p?.lowStockThreshold ?? 3}" /></label>
        <label>Proveedor
          <select name="supplierId"><option value="">—</option>${supplierOptions}
</select>
        </label>
      </div>
      <p class="error-text" hidden></p>
      <button type="submit" class="btn btn-primary btn-block">${p ? 'Guardar
cambios' : 'Crear prenda'}</button>
    </form>
  `);

  form.addEventListener('submit', async (e) => {
    e.preventDefault();
    const fd = new FormData(form);
    const payload = Object.fromEntries(fd.entries());
    try {
      if (p) await api(`/api/products/${p.id}`, { method: 'PUT', body: payload });
      else await api('/api/products', { method: 'POST', body: payload });
      closeModal();
      loadStock();
    } catch (err) {
      const errEl = form.querySelector('.error-text');
      errEl.textContent = err.message;
      errEl.hidden = false;
    }
  });

  showModal(p ? 'Editar prenda' : 'Nueva prenda', form);
}

// ---------- VENTA DIARIA ----------
function todayStr() {
  const d = new Date();
  const tz = d.getTimezoneOffset() * 60000;
  return new Date(d - tz).toISOString().slice(0, 10);
}

let saleItemRowId = 0;
```

```
function addSaleItemRow() {
  const container = document.getElementById('sale-items');
  const rowId = ++saleItemRowId;
  const row = el(`
    <div class="sale-item-row" data-row-id="${rowId}">
      <div class="sale-item-picker">
        <input type="text" class="sale-item-search" placeholder="Buscar prenda,
pieza o conjunto..." autocomplete="off" />
        <div class="sale-item-results" hidden></div>
      </div>
      <input type="number" class="sale-item-qty" min="1" value="1" />
      <input type="number" class="sale-item-price" step="0.01" />
      <span class="sale-item-subtotal">$0</span>
      <button type="button" class="btn btn-ghost btn-sm sale-item-remove"
title="Quitar prenda">×</button>
    </div>
  `);
  container.appendChild(row);
  updateSaleRowSubtotal(row);
  updateSaleRemoveButtonsState();
  return row;
}

// Busca por substring en cualquier parte del texto (no solo desde el
// principio) y, si una prenda encontrada pertenece a un conjunto, trae
// también al resto de sus piezas agrupadas, para poder elegir justo la que
// se vendió (ej: buscar "campera" y poder elegir solo esa, dejando el short
// del mismo conjunto sin tocar).
function searchSaleProducts(query) {
  const q = query.trim().toLowerCase();
  let pool = state.products;
  if (q) {
    const direct = state.products.filter((p) =>
      p.name.toLowerCase().includes(q) ||
      (p.category || '').toLowerCase().includes(q) ||
      (p.piece || '').toLowerCase().includes(q) ||
      (p.size || '').toLowerCase().includes(q) ||
      (p.color || '').toLowerCase().includes(q) ||
      (p.setName || '').toLowerCase().includes(q)
    );
    const setNames = new Set(direct.filter((p) => p.setName).map((p) =>
p.setName));
    pool = state.products.filter((p) => direct.includes(p) || (p.setName &&
setNames.has(p.setName)));
  }
  return pool.slice(0, 40);
}
```

```
// Arma "Talle M · Negro" (o solo la parte que tenga cargada) para que se
// pueda distinguir de un vistazo entre talles/colores de la misma prenda.
function saleProductDetail(p) {
  const parts = [];
  if (p.size) parts.push(`Talle ${p.size}`);
  if (p.color) parts.push(p.color);
  return parts.join(' · ');
}

// Etiqueta de cada fila dentro del listado de resultados.
function saleResultRowLabel(p) {
  const base = p.piece || p.name;
  const detail = saleProductDetail(p);
  return detail ? `${base} — ${detail}` : base;
}

// Etiqueta que queda escrita en el buscador una vez que se elige la prenda
// (acá sí hace falta repetir el nombre del conjunto, porque el encabezado
// del grupo ya no se ve una vez cerrado el listado).
function saleSelectedLabel(p) {
  const base = p.setName ? `${p.setName} — ${p.piece || p.name}` : p.name;
  const detail = saleProductDetail(p);
  return detail ? `${base} — ${detail}` : base;
}

function renderSaleResults(row, query) {
  const resultsEl = row.querySelector('.sale-item-results');
  const pool = searchSaleProducts(query);

  if (pool.length === 0) {
    resultsEl.innerHTML = '<div class="sale-item-result-empty">No se encontraron
prendas.</div>';
    resultsEl.hidden = false;
    return;
  }

  const groups = new Map();
  const standalone = [];
  pool.forEach((p) => {
    if (p.setName) {
      if (!groups.has(p.setName)) groups.set(p.setName, []);
      groups.get(p.setName).push(p);
    } else {
      standalone.push(p);
    }
  });
```

```
  let html = '';
  groups.forEach((items, setName) => {
    html += `<div class="sale-item-result-group-label">${setName}</div>`;
    items.forEach((p) => {
      html += `<div class="sale-item-result" data-pid="${p.id}">
<span>${saleResultRowLabel(p)}</span><span class="sale-item-result-stock">stock:
${p.stock}</span></div>`;
    });
  });
  standalone.forEach((p) => {
    html += `<div class="sale-item-result" data-pid="${p.id}">
<span>${saleResultRowLabel(p)}</span><span class="sale-item-result-stock">stock:
${p.stock}</span></div>`;
  });

  resultsEl.innerHTML = html;
  resultsEl.hidden = false;
}

function selectSaleProduct(row, productId) {
  const p = state.products.find((x) => x.id === productId);
  if (!p) return;
  row.dataset.productId = productId;
  row.querySelector('.sale-item-search').value = saleSelectedLabel(p);
  row.querySelector('.sale-item-price').value = p.salePrice;
  row.querySelector('.sale-item-results').hidden = true;
  updateSaleRowSubtotal(row);
  updateSaleTotalPreview();
}

function updateSaleRemoveButtonsState() {
  const rows = document.querySelectorAll('#sale-items .sale-item-row');
  rows.forEach((r) => {
    r.querySelector('.sale-item-remove').disabled = rows.length <= 1;
  });
}

function updateSaleRowSubtotal(row) {
  const qty = Number(row.querySelector('.sale-item-qty').value) || 0;
  const price = Number(row.querySelector('.sale-item-price').value) || 0;
  row.querySelector('.sale-item-subtotal').textContent = money(qty * price);
}

document.getElementById('sale-add-item-btn').addEventListener('click', () =>
addSaleItemRow());
```

```
const saleItemsContainer = document.getElementById('sale-items');

saleItemsContainer.addEventListener('focus', (e) => {
  if (e.target.classList.contains('sale-item-search')) {
    renderSaleResults(e.target.closest('.sale-item-row'), e.target.value);
  }
}, true);

saleItemsContainer.addEventListener('mousedown', (e) => {
  if (e.target.closest('.sale-item-results')) e.preventDefault();
});

saleItemsContainer.addEventListener('blur', (e) => {
  if (e.target.classList.contains('sale-item-search')) {
    const row = e.target.closest('.sale-item-row');
    setTimeout(() => { row.querySelector('.sale-item-results').hidden = true; },
150);
  }
}, true);

saleItemsContainer.addEventListener('input', (e) => {
  if (e.target.classList.contains('sale-item-search')) {
    const row = e.target.closest('.sale-item-row');
    delete row.dataset.productId;
    renderSaleResults(row, e.target.value);
  }
  if (e.target.classList.contains('sale-item-qty') ||
e.target.classList.contains('sale-item-price')) {
    const row = e.target.closest('.sale-item-row');
    updateSaleRowSubtotal(row);
    updateSaleTotalPreview();
  }
});

saleItemsContainer.addEventListener('click', (e) => {
  const resultEl = e.target.closest('.sale-item-result');
  if (resultEl) {
    selectSaleProduct(resultEl.closest('.sale-item-row'),
Number(resultEl.dataset.pid));
    return;
  }
  if (e.target.classList.contains('sale-item-remove')) {
    const rows = document.querySelectorAll('#sale-items .sale-item-row');
    if (rows.length <= 1) return;
    e.target.closest('.sale-item-row').remove();
    updateSaleRemoveButtonsState();
    updateSaleTotalPreview();
```

```
  }
});

async function loadVentaView() {
  await refreshProducts();
  const dateInput = document.getElementById('sale-date');
  if (!dateInput.value) dateInput.value = todayStr();

  document.getElementById('sale-items').innerHTML = '';
  saleItemRowId = 0;
  addSaleItemRow();

  state.clients = await api('/api/clients');
  const clientSelect = document.getElementById('sale-client');
  clientSelect.innerHTML = state.clients.map((c) => `<option
value="${c.id}">${c.name}</option>`).join('');

  updateSaleTotalPreview();
}

['sale-discount-type', 'sale-discount-value'].forEach((id) => {
  document.getElementById(id).addEventListener('input', updateSaleTotalPreview);
});

function updateSaleTotalPreview() {
  let subtotal = 0;
  document.querySelectorAll('#sale-items .sale-item-row').forEach((row) => {
    const qty = Number(row.querySelector('.sale-item-qty').value) || 0;
    const price = Number(row.querySelector('.sale-item-price').value) || 0;
    subtotal += qty * price;
  });
  const discType = document.getElementById('sale-discount-type').value;
  const discVal = Number(document.getElementById('sale-discount-value').value) ||
0;
  let discAmt = 0;
  if (discType === 'percent') discAmt = (subtotal * discVal) / 100;
  else if (discType === 'amount') discAmt = discVal;
  const total = Math.max(0, subtotal - discAmt);
  document.getElementById('sale-total-preview').textContent = money(total);
}

document.getElementById('sale-is-cc').addEventListener('change', (e) => {
  document.getElementById('sale-payment-wrap').hidden = e.target.checked;
  document.getElementById('sale-client-wrap').hidden = !e.target.checked;
});

document.getElementById('sale-client-new-btn').addEventListener('click', () =>
```

```
openClientForm(null, true));

document.getElementById('sale-form').addEventListener('submit', async (e) => {
  e.preventDefault();
  const errEl = document.getElementById('sale-error');
  errEl.hidden = true;
  const isCC = document.getElementById('sale-is-cc').checked;

  const items = Array.from(document.querySelectorAll('#sale-items .sale-item-
row')).map((row) => ({
    productId: row.dataset.productId ? Number(row.dataset.productId) : null,
    qty: Number(row.querySelector('.sale-item-qty').value),
    unitPrice: Number(row.querySelector('.sale-item-price').value)
  }));

  if (items.length === 0 || items.some((it) => !it.productId || !it.qty || it.qty
<= 0)) {
    errEl.textContent = 'Revisá que todas las prendas tengan cantidad y precio
cargados.';
    errEl.hidden = false;
    return;
  }

  const payload = {
    items,
    discountType: document.getElementById('sale-discount-type').value,
    discountValue: Number(document.getElementById('sale-discount-value').value) ||
0,
    isCuentaCorriente: isCC,
    paymentMethod: document.getElementById('sale-payment').value,
    clientId: isCC ? Number(document.getElementById('sale-client').value) : null,
    note: document.getElementById('sale-note').value,
    date: document.getElementById('sale-date').value
  };
  try {
    await api('/api/sales/batch', { method: 'POST', body: payload });
    const keepDate = document.getElementById('sale-date').value;
    document.getElementById('sale-form').reset();
    document.getElementById('sale-date').value = keepDate;
    document.getElementById('sale-is-cc').checked = false;
    document.getElementById('sale-payment-wrap').hidden = false;
    document.getElementById('sale-client-wrap').hidden = true;
    await loadVentaView();
    alert(`Venta registrada ✔ (${items.length} prenda${items.length > 1 ? 's' :
''})`);
  } catch (err) {
    errEl.textContent = err.message;
```

```
    errEl.hidden = false;
  }
});

// ---------- HISTORIAL DE VENTAS ----------
async function loadHistorial(params = {}) {
  const qs = new URLSearchParams(params).toString();
  const sales = await api('/api/sales' + (qs ? '?' + qs : ''));
  const isOwner = state.user.role === 'duena';
  const tbody = document.querySelector('#sales-table tbody');
  tbody.innerHTML = '';

  // Las prendas cargadas juntas en una misma venta comparten groupId: se
  // muestran agrupadas (fecha/pago/vendedora una sola vez) con un renglón
  // de total al final del grupo.
  let i = 0;
  while (i < sales.length) {
    const group = [sales[i]];
    while (i + 1 < sales.length && sales[i + 1].groupId && sales[i + 1].groupId
=== sales[i].groupId) {
      group.push(sales[++i]);
    }
    i++;

    group.forEach((s, idx) => {
      const row = el(`<tr class="${group.length > 1 ? 'grouped-row' : ''}">
        <td>${idx === 0 ? fmtDate(s.date) : ''}</td>
        <td>${s.productName}</td>
        <td>${s.qty}</td>
        <td>${money(s.unitPrice)}</td>
        <td>${money(s.discountAmt)}</td>
        <td>${money(s.total)}</td>
        <td>${idx === 0 ? (PAYMENT_LABELS[s.paymentMethod] || s.paymentMethod) +
(s.clientName ? ' (' + s.clientName + ')' : '') : ''}</td>
        <td>${idx === 0 ? s.employeeName : ''}</td>
        <td>${isOwner ? `<button class="btn btn-ghost btn-sm" data-
edit="${s.id}">Editar</button> <button class="btn btn-danger btn-sm" data-
del="${s.id}">Borrar</button>` : ''}</td>
      </tr>`);
      tbody.appendChild(row);
    });

    if (group.length > 1) {
      const groupTotal = group.reduce((s, x) => s + x.total, 0);
      tbody.appendChild(el(`<tr class="group-total-row">
        <td colspan="5" style="text-align:right;">Total de la venta
(${group.length} prendas):</td>
```

```
        <td><strong>${money(groupTotal)}</strong></td>
        <td colspan="3"></td>
      </tr>`));
    }
  }

  if (isOwner) {
    tbody.querySelectorAll('[data-del]').forEach((b) =>
      b.addEventListener('click', async () => {
        if (!confirm('¿Borrar esta venta? Se repone el stock.')) return;
        await api(`/api/sales/${b.dataset.del}`, { method: 'DELETE' });
        loadHistorial();
      })
    );
    tbody.querySelectorAll('[data-edit]').forEach((b) =>
      b.addEventListener('click', () => openSaleEditForm(sales.find((s) => s.id
=== Number(b.dataset.edit))))
    );
  }
}

function openSaleEditForm(sale) {
  const form = el(`
    <form class="form-card" style="max-width:none;box-shadow:none;padding:0;">
      <p style="margin-top:0;color:var(--gray);font-
size:13px;">${sale.productName}</p>
      <div class="grid-2">
        <label>Cantidad <input name="qty" type="number" min="1"
value="${sale.qty}" required /></label>
        <label>Precio unitario <input name="unitPrice" type="number" step="0.01"
min="0" value="${sale.unitPrice}" required /></label>
      </div>
      <label>Fecha de la venta <input name="date" type="date"
value="${sale.date.slice(0, 10)}" required /></label>
      <div class="grid-2">
        <label>Descuento
          <select name="discountType">
            <option value="none" ${sale.discountAmt === 0 ? 'selected' : ''}>Sin
descuento</option>
            <option value="amount">Monto fijo ($)</option>
          </select>
        </label>
        <label>Valor descuento <input name="discountValue" type="number"
step="0.01" value="${sale.discountAmt}" /></label>
      </div>
      <label>Medio de pago
        <select name="paymentMethod" ${sale.paymentMethod === 'cuenta_corriente' ?
```

```
'disabled' : ''}>
          <option value="efectivo" ${sale.paymentMethod === 'efectivo' ?
'selected' : ''}>Efectivo</option>
          <option value="debito" ${sale.paymentMethod === 'debito' ? 'selected' :
''}>Tarjeta débito</option>
          <option value="credito" ${sale.paymentMethod === 'credito' ? 'selected'
: ''}>Tarjeta crédito</option>
          <option value="transferencia" ${sale.paymentMethod === 'transferencia' ?
'selected' : ''}>Transferencia</option>
          <option value="cuenta_corriente" ${sale.paymentMethod ===
'cuenta_corriente' ? 'selected' : ''}>Cuenta corriente (no editable acá)</option>
        </select>
      </label>
      <label>Nota <input name="note" value="${sale.note || ''}" /></label>
      <p class="error-text" hidden></p>
      <button type="submit" class="btn btn-primary btn-block">Guardar
cambios</button>
    </form>
  `);
  form.addEventListener('submit', async (e) => {
    e.preventDefault();
    const payload = Object.fromEntries(new FormData(form).entries());
    try {
      await api(`/api/sales/${sale.id}`, { method: 'PUT', body: payload });
      closeModal();
      loadHistorial();
    } catch (err) {
      const errEl = form.querySelector('.error-text');
      errEl.textContent = err.message;
      errEl.hidden = false;
    }
  });
  showModal('Editar venta', form);
}

document.getElementById('hist-filter-btn').addEventListener('click', () => {
  loadHistorial({ from: document.getElementById('hist-from').value, to:
document.getElementById('hist-to').value });
});

// ---------- PROVEEDORES ----------
async function loadSuppliers() {
  state.suppliers = await api('/api/suppliers');
  const tbody = document.querySelector('#suppliers-table tbody');
  tbody.innerHTML = '';
  state.suppliers.forEach((s) => {
    tbody.appendChild(el(`<tr>
```

```
      <td>${s.name}</td><td>${s.phone || '—'}</td><td>${s.notes || '—'}</td>
      <td>
        <button class="btn btn-ghost btn-sm" data-edit="${s.id}">Editar</button>
        <button class="btn btn-danger btn-sm" data-del="${s.id}">Borrar</button>
      </td>
    </tr>`));
  });
  tbody.querySelectorAll('[data-edit]').forEach((b) => b.addEventListener('click',
() => openSupplierForm(Number(b.dataset.edit))));
  tbody.querySelectorAll('[data-del]').forEach((b) =>
    b.addEventListener('click', async () => {
      if (!confirm('¿Borrar este proveedor?')) return;
      await api(`/api/suppliers/${b.dataset.del}`, { method: 'DELETE' });
      loadSuppliers();
    })
  );
}

document.getElementById('new-supplier-btn').addEventListener('click', () =>
openSupplierForm(null));

function openSupplierForm(id) {
  const s = id ? state.suppliers.find((x) => x.id === id) : null;
  const form = el(`
    <form class="form-card" style="max-width:none;box-shadow:none;padding:0;">
      <label>Nombre <input name="name" required value="${s?.name || ''}" />
</label>
      <label>Teléfono <input name="phone" value="${s?.phone || ''}" /></label>
      <label>Notas <input name="notes" value="${s?.notes || ''}" /></label>
      <button type="submit" class="btn btn-primary btn-block">${s ? 'Guardar' :
'Crear'}</button>
    </form>
  `);
  form.addEventListener('submit', async (e) => {
    e.preventDefault();
    const payload = Object.fromEntries(new FormData(form).entries());
    if (s) await api(`/api/suppliers/${s.id}`, { method: 'PUT', body: payload });
    else await api('/api/suppliers', { method: 'POST', body: payload });
    closeModal();
    loadSuppliers();
  });
  showModal(s ? 'Editar proveedor' : 'Nuevo proveedor', form);
}

// ---------- MOVIMIENTOS DE STOCK ----------
async function loadMovimientos() {
  await refreshProducts();
```

```
  const select = document.getElementById('mov-product');
  select.innerHTML = state.products.map((p) => `<option
value="${p.id}">${p.name}${p.size ? ' - ' + p.size : ''}</option>`).join('');

  const totalCosto = state.products.reduce((s, p) => s + (Number(p.costPrice) ||
0) * (Number(p.stock) || 0), 0);
  const totalVenta = state.products.reduce((s, p) => s + (Number(p.salePrice) ||
0) * (Number(p.stock) || 0), 0);
  const valueCards = document.getElementById('stock-value-cards');
  valueCards.innerHTML = '';
  valueCards.appendChild(cardEl('Stock sin vender (a precio costo)',
money(totalCosto)));
  valueCards.appendChild(cardEl('Stock sin vender (a precio de venta)',
money(totalVenta)));

  const movements = await api('/api/movements');
  const tbody = document.querySelector('#movements-table tbody');
  tbody.innerHTML = '';
  movements.forEach((m) => {
    tbody.appendChild(el(`<tr>
      <td>${fmtDate(m.date)}</td><td>${m.productName}</td>
      <td>${m.type === 'ingreso' ? 'Ingreso' : 'Egreso'}</td><td>${m.qty}</td>
<td>${m.reason || '—'}</td>
    </tr>`));
  });
}

document.getElementById('mov-type').addEventListener('change', (e) => {
  document.getElementById('mov-expense-wrap').hidden = e.target.value !==
'ingreso';
});

document.getElementById('movement-form').addEventListener('submit', async (e) => {
  e.preventDefault();
  const errEl = document.getElementById('movement-error');
  errEl.hidden = true;
  const payload = {
    productId: Number(document.getElementById('mov-product').value),
    type: document.getElementById('mov-type').value,
    qty: Number(document.getElementById('mov-qty').value),
    reason: document.getElementById('mov-reason').value,
    registerAsExpense: document.getElementById('mov-register-expense').checked
  };
  try {
    await api('/api/movements', { method: 'POST', body: payload });
    document.getElementById('movement-form').reset();
    loadMovimientos();
```

```
  } catch (err) {
    errEl.textContent = err.message;
    errEl.hidden = false;
  }
});

// ---------- GASTOS ----------
async function loadGastos() {
  const expenses = await api('/api/expenses');
  const tbody = document.querySelector('#expenses-table tbody');
  tbody.innerHTML = '';
  expenses.forEach((ex) => {
    tbody.appendChild(el(`<tr>
      <td>${fmtDate(ex.date)}</td><td>${ex.category}</td><td>${ex.description ||
'—'}</td><td>${money(ex.amount)}</td>
      <td><button class="btn btn-danger btn-sm" data-
del="${ex.id}">Borrar</button></td>
    </tr>`));
  });
  tbody.querySelectorAll('[data-del]').forEach((b) =>
    b.addEventListener('click', async () => {
      if (!confirm('¿Borrar este gasto?')) return;
      await api(`/api/expenses/${b.dataset.del}`, { method: 'DELETE' });
      loadGastos();
    })
  );
}

document.getElementById('expense-form').addEventListener('submit', async (e) => {
  e.preventDefault();
  const payload = {
    category: document.getElementById('expense-category').value,
    description: document.getElementById('expense-description').value,
    amount: Number(document.getElementById('expense-amount').value)
  };
  await api('/api/expenses', { method: 'POST', body: payload });
  document.getElementById('expense-form').reset();
  loadGastos();
});

// ---------- CUENTAS CORRIENTES ----------
async function loadClientes() {
  state.clients = await api('/api/clients');
  const tbody = document.querySelector('#clients-table tbody');
  tbody.innerHTML = '';
  state.clients.forEach((c) => {
    tbody.appendChild(el(`<tr>
```

```
      <td>${c.name}</td><td>${c.phone || '—'}</td><td>${money(c.balance)}</td>
      <td><button class="btn btn-ghost btn-sm" data-open="${c.id}">Ver
cuenta</button></td>
    </tr>`));
  });
  tbody.querySelectorAll('[data-open]').forEach((b) => b.addEventListener('click',
() => openClientDetail(Number(b.dataset.open))));
}

document.getElementById('new-client-btn').addEventListener('click', () =>
openClientForm(null, false));

function openClientForm(id, forSaleForm) {
  const form = el(`
    <form class="form-card" style="max-width:none;box-shadow:none;padding:0;">
      <label>Nombre <input name="name" required /></label>
      <label>Teléfono <input name="phone" /></label>
      <button type="submit" class="btn btn-primary btn-block">Crear
clienta</button>
    </form>
  `);
  form.addEventListener('submit', async (e) => {
    e.preventDefault();
    const payload = Object.fromEntries(new FormData(form).entries());
    const client = await api('/api/clients', { method: 'POST', body: payload });
    closeModal();
    if (forSaleForm) {
      state.clients.push(client);
      const clientSelect = document.getElementById('sale-client');
      clientSelect.innerHTML = state.clients.map((c) => `<option
value="${c.id}">${c.name}</option>`).join('');
      clientSelect.value = client.id;
    } else {
      loadClientes();
    }
  });
  showModal('Nueva clienta', form);
}

async function openClientDetail(id) {
  state.currentClientId = id;
  showView('cc-detail');
  await refreshClientDetail();
}

async function refreshClientDetail() {
  const { client, balance, movements } = await
```

```
api(`/api/clients/${state.currentClientId}/ledger`);
  document.getElementById('cc-detail-name').textContent = client.name;
  document.getElementById('cc-detail-balance').textContent = money(balance);
  const isOwner = state.user.role === 'duena';
  const tbody = document.querySelector('#cc-ledger-table tbody');
  tbody.innerHTML = '';
  movements.forEach((m) => {
    tbody.appendChild(el(`<tr>
      <td>${fmtDate(m.date)}</td>
      <td>${m.kind === 'cargo' ? m.productName : 'Pago (' +
(PAYMENT_LABELS[m.paymentMethod] || m.paymentMethod) + ')'}</td>
      <td>${m.kind === 'cargo' ? money(m.amount) : ''}</td>
      <td>${m.kind === 'pago' ? money(m.amount) : ''}</td>
      <td>${isOwner && m.kind === 'pago' ? `<button class="btn btn-danger btn-sm"
data-delpay="${m.id}">Borrar</button>` : ''}</td>
    </tr>`));
  });
  tbody.querySelectorAll('[data-delpay]').forEach((b) =>
    b.addEventListener('click', async () => {
      if (!confirm('¿Borrar este pago?')) return;
      await api(`/api/payments/${b.dataset.delpay}`, { method: 'DELETE' });
      refreshClientDetail();
    })
  );
}

document.getElementById('cc-detail-back').addEventListener('click', () =>
showView('cc'));

document.getElementById('cc-payment-form').addEventListener('submit', async (e) =>
{
  e.preventDefault();
  const payload = {
    amount: Number(document.getElementById('cc-payment-amount').value),
    paymentMethod: document.getElementById('cc-payment-method').value
  };
  await api(`/api/clients/${state.currentClientId}/payments`, { method: 'POST',
body: payload });
  document.getElementById('cc-payment-form').reset();
  refreshClientDetail();
});

// ---------- REPORTES ----------
async function loadReportes(params = {}) {
  const qs = new URLSearchParams(params).toString();
  const r = await api('/api/reports/summary' + (qs ? '?' + qs : ''));
  const cards = document.getElementById('report-cards');
```

```
  cards.innerHTML = '';
  cards.appendChild(cardEl('Total ventas', money(r.totalVentas)));
  cards.appendChild(cardEl('Descuentos otorgados', money(r.totalDescuentos)));
  cards.appendChild(cardEl('Costo de lo vendido', money(r.totalCosto)));
  cards.appendChild(cardEl('Ganancia bruta', money(r.gananciaBruta)));
  cards.appendChild(cardEl('Gastos generales', money(r.totalGastos)));
  cards.appendChild(cardEl('Ganancia neta', money(r.gananciaNeta)));
  cards.appendChild(cardEl('Cuentas corrientes pendientes', money(r.pendingCC)));
  cards.appendChild(cardEl('Cantidad de ventas', r.cantidadVentas));

  const payTbody = document.querySelector('#report-payment-table tbody');
  payTbody.innerHTML = '';
  Object.entries(r.porMedioPago).forEach(([method, total]) => {
    payTbody.appendChild(el(`<tr><td>${PAYMENT_LABELS[method] || method}</td>
<td>${money(total)}</td></tr>`));
  });

  const topTbody = document.querySelector('#report-top-table tbody');
  topTbody.innerHTML = '';
  r.topProductos.forEach((p) => {
    topTbody.appendChild(el(`<tr><td>${p.name}</td><td>${p.unidades} un.</td>
<td>${money(p.total)}</td></tr>`));
  });
}

document.getElementById('rep-filter-btn').addEventListener('click', () => {
  loadReportes({ from: document.getElementById('rep-from').value, to:
document.getElementById('rep-to').value });
});

// ---------- USUARIAS ----------
async function loadUsuarios() {
  const users = await api('/api/users');
  const tbody = document.querySelector('#users-table tbody');
  tbody.innerHTML = '';
  users.forEach((u) => {
    tbody.appendChild(el(`<tr>
      <td>${u.username}</td><td>${u.name}</td><td>${u.role === 'duena' ? 'Dueña' :
'Empleada'}</td>
      <td>
        <button class="btn btn-ghost btn-sm" data-edit="${u.id}">Editar</button>
        ${u.id !== state.user.id ? `<button class="btn btn-danger btn-sm" data-
del="${u.id}">Borrar</button>` : ''}
      </td>
    </tr>`));
  });
  tbody.querySelectorAll('[data-edit]').forEach((b) => b.addEventListener('click',
```

```
() => openUserForm(users.find((u) => u.id === Number(b.dataset.edit)))));
  tbody.querySelectorAll('[data-del]').forEach((b) =>
    b.addEventListener('click', async () => {
      if (!confirm('¿Borrar esta usuaria?')) return;
      await api(`/api/users/${b.dataset.del}`, { method: 'DELETE' });
      loadUsuarios();
    })
  );
}

document.getElementById('new-user-btn').addEventListener('click', () =>
openUserForm(null));

function openUserForm(u) {
  const form = el(`
    <form class="form-card" style="max-width:none;box-shadow:none;padding:0;">
      <label>Nombre <input name="name" required value="${u?.name || ''}" />
</label>
      ${u ? '' : '<label>Usuario <input name="username" required /></label>'}
      <label>Rol
        <select name="role">
          <option value="empleada" ${u?.role === 'empleada' ? 'selected' :
''}>Empleada</option>
          <option value="duena" ${u?.role === 'duena' ? 'selected' :
''}>Dueña</option>
        </select>
      </label>
      <label>${u ? 'Nueva contraseña (opcional)' : 'Contraseña'} <input
name="password" type="password" ${u ? '' : 'required'} /></label>
      <p class="error-text" hidden></p>
      <button type="submit" class="btn btn-primary btn-block">${u ? 'Guardar' :
'Crear usuaria'}</button>
    </form>
  `);
  form.addEventListener('submit', async (e) => {
    e.preventDefault();
    const payload = Object.fromEntries(new FormData(form).entries());
    try {
      if (u) await api(`/api/users/${u.id}`, { method: 'PUT', body: payload });
      else await api('/api/users', { method: 'POST', body: payload });
      closeModal();
      loadUsuarios();
    } catch (err) {
      const errEl = form.querySelector('.error-text');
      errEl.textContent = err.message;
      errEl.hidden = false;
    }
```

```
  });
  showModal(u ? 'Editar usuaria' : 'Nueva usuaria', form);
}

// ---------- init ----------
checkSession();
```